



Hands-on Exercise 3b: Programming Animated Statistical Graphics with R

AUTHOR

PHAM Hung Son

PUBLISHED

January 26, 2026

MODIFIED

January 26, 2026

1 Learning Outcome

In this hands-on exercise, you will learn how to create animated data visualisation by using **gganimate** and **plotly r** packages. At the same time, you will also learn how to (i) reshape data by using **tidyr** package, and (ii) process, wrangle and transform data by using **dplyr** package.

2 Getting started

2.1 Installing and loading the required libraries

First, we write code chunk to install and load the following R packages:

- [plotly](#), R library for plotting interactive statistical graphs.
- [gganimate](#), an ggplot extension for creating animated statistical graphs.
- [gifski](#) converts video frames to GIF animations using pngquant's fancy features for efficient cross-frame palettes and temporal dithering. It produces animated GIFs that use thousands of colors per frame.
- [gapminder](#): An excerpt of the data available at Gapminder.org. We just want to use its *country_colors* scheme.
- [tidyverse](#), a family of modern R packages specially designed to support data science, analysis and communication task including creating static statistical graphs.

```
pacman::p_load(readxl, gifski, gapminder, plotly, gganimate, tidyverse)
```

2.2 Importing the data

In this hands-on exercise, the *Data* worksheet from *GlobalPopulation* Excel workbook will be used.

```
col <- c("Country", "Continent")
globalPop <- read_xls("data/GlobalPopulation.xls",
                     sheet="Data") %>%
  mutate_each(funs(factor(.)), col) %>%
  mutate(Year = as.integer(Year))
```

Things to learn from the code chunk above

- [read_xls\(\)](#) of **readxl** package is used to import the Excel worksheet.
- [mutate_each\(\)](#) of dplyr package is used to convert all character data type into factor.
- [mutate](#) of dplyr package is used to convert data values of Year field into integer.

Unfortunately, [mutate_each\(\)](#) was deprecated in dplyr 0.7.0. and [funs\(\)](#) was deprecated in dplyr 0.8.0. In view of this, we will re-write the code by using [mutate_at\(\)](#) as shown in the code chunk below.

```
col <- c("Country", "Continent")
globalPop <- read_xls("data/GlobalPopulation.xls",
                     sheet="Data") %>%
  mutate_at(col, as.factor) %>%
  mutate(Year = as.integer(Year))
```

Instead of using [mutate_at\(\)](#), [across\(\)](#) can be used to derive the same outputs.

```
col <- c("Country", "Continent")
globalPop <- read_xls("data/GlobalPopulation.xls",
                     sheet="Data") %>%
  mutate(across(col, as.factor)) %>%
  mutate(Year = as.integer(Year))
```

3 Animated Data Visualisation: ganimate methods

[gganimate](#) extends the grammar of graphics as implemented by ggplot2 to include the description of animation. It does this by providing a range of new grammar classes that can be added to the plot object in order to customise how it should change with time.

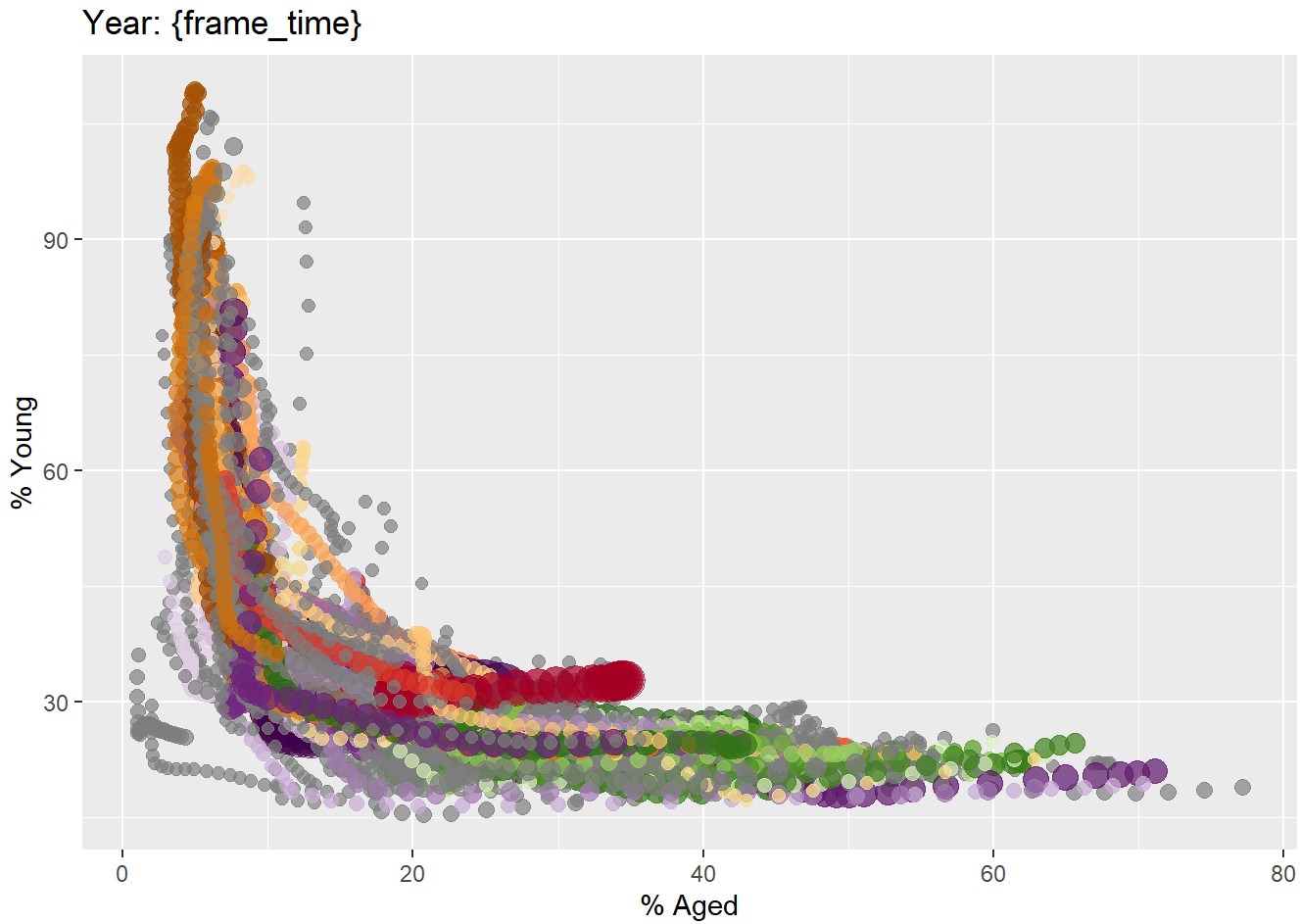
- [transition_*\(\)](#) defines how the data should be spread out and how it relates to itself across time.
- [view_*\(\)](#) defines how the positional scales should change along the animation.
- [shadow_*\(\)](#) defines how data from other points in time should be presented in the given point in time.
- [enter_*\(\)/exit_*\(\)](#) defines how new data should appear and how old data should disappear during the course of the animation.

- `ease_aes()` defines how different aesthetics should be eased during transitions.

3.1 Building a static population bubble plot

In the code chunk below, the basic ggplot2 functions are used to create a static bubble plot.

```
ggplot(globalPop, aes(x = Old, y = Young,
                     size = Population,
                     colour = Country)) +
  geom_point(alpha = 0.7,
             show.legend = FALSE) +
  scale_colour_manual(values = country_colors) +
  scale_size(range = c(2, 12)) +
  labs(title = 'Year: {frame_time}',
       x = '% Aged',
       y = '% Young')
```

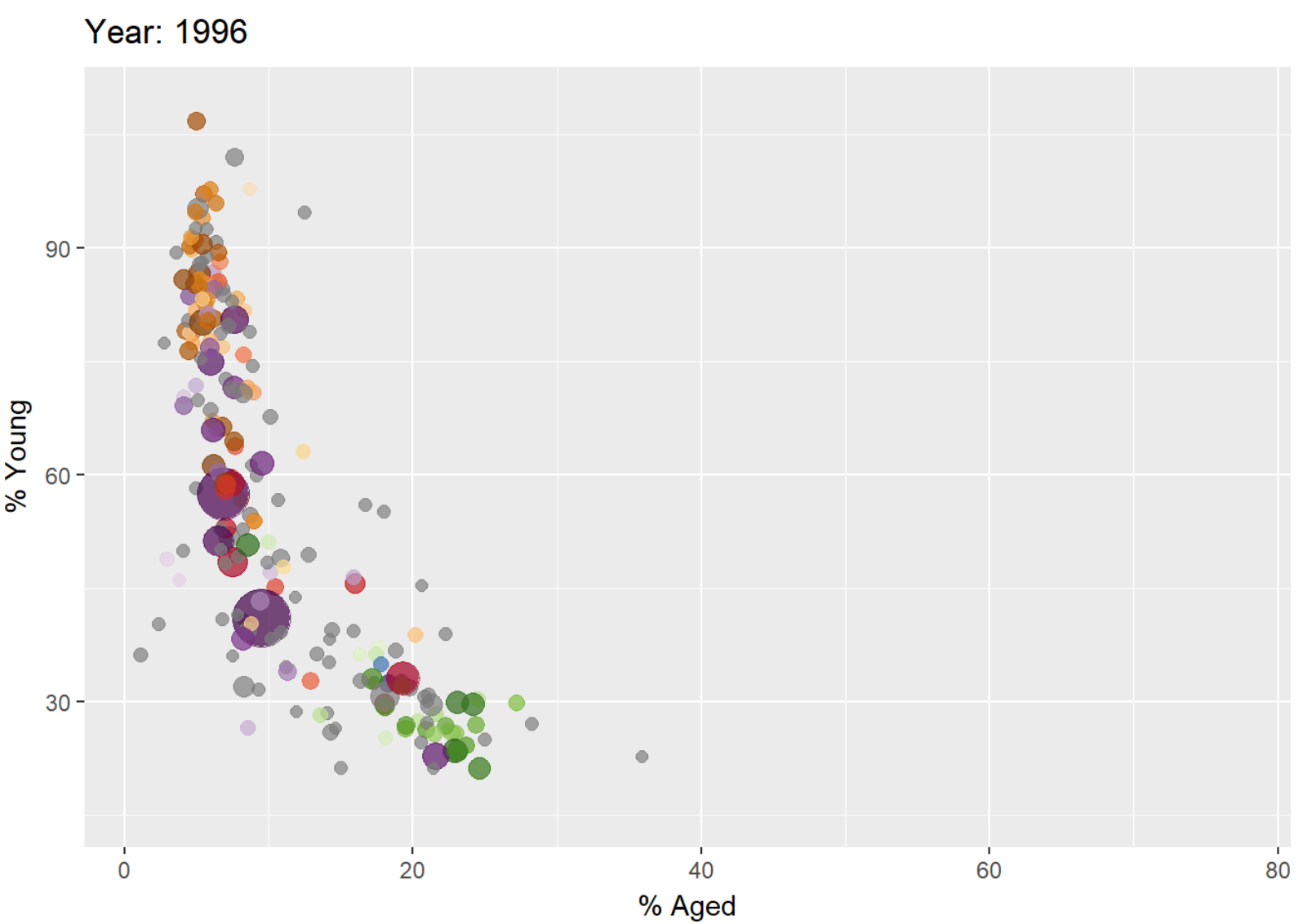


3.2 Building the animated bubble plot

In the code chunk below,

- `transition_time()` of **gganimate** is used to create transition through distinct states in time (i.e. Year).
- `ease_aes()` is used to control easing of aesthetics. The default is `linear`. Other methods are: quadratic, cubic, quartic, quintic, sine, circular, exponential, elastic, back, and bounce.

```
ggplot(globalPop, aes(x = Old, y = Young,
                     size = Population,
                     colour = Country)) +
  geom_point(alpha = 0.7,
            show.legend = FALSE) +
  scale_colour_manual(values = country_colors) +
  scale_size(range = c(2, 12)) +
  labs(title = 'Year: {frame_time}',
       x = '% Aged',
       y = '% Young') +
  transition_time(Year) +
  ease_aes('linear')
```



4 Animated Data Visualisation: plotly

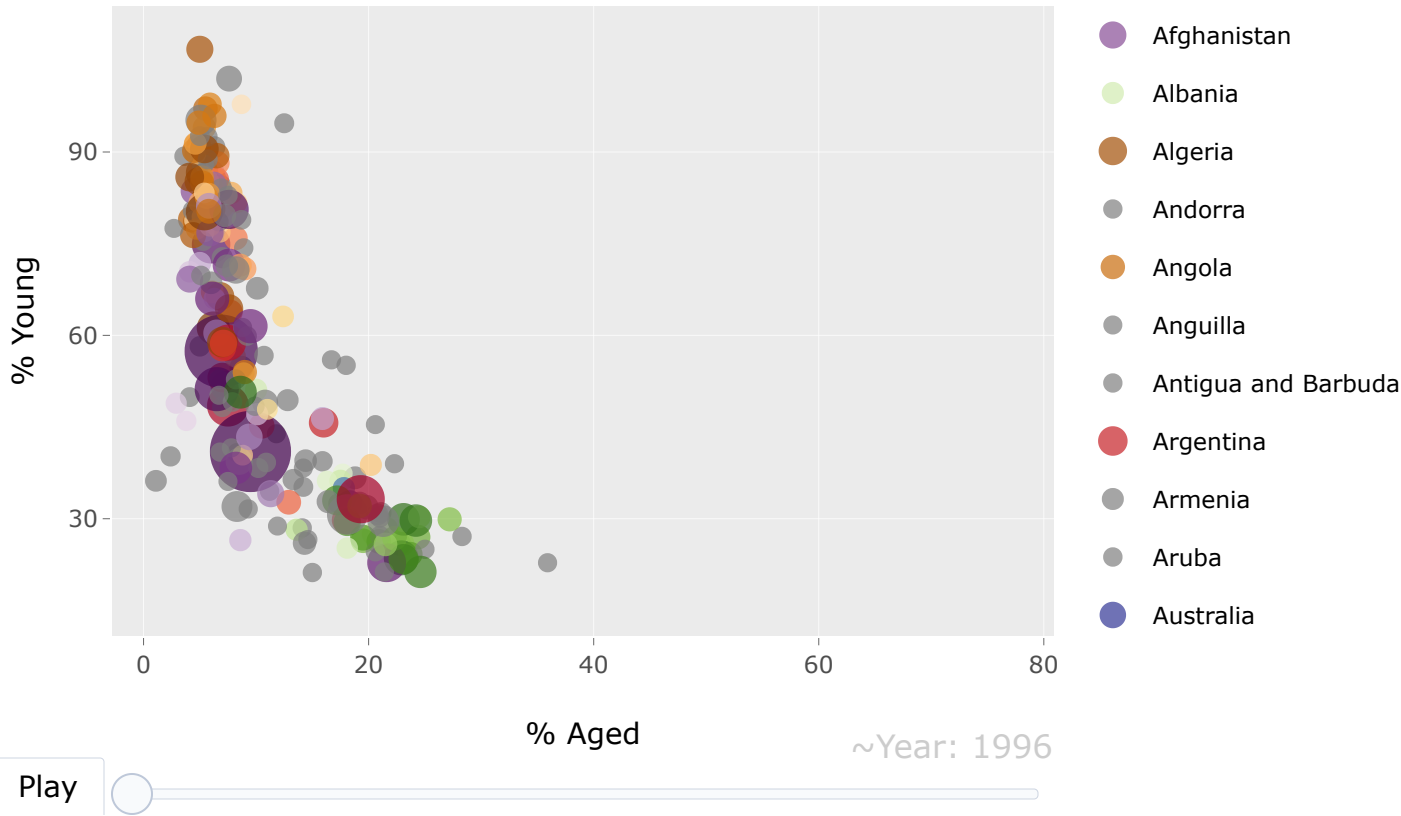
In **Plotly R** package, both `ggplotly()` and `plot_ly()` support key frame animations through the `frame` argument/aesthetic. They also support an `ids` argument/aesthetic to ensure smooth transitions between objects with the same id (which helps facilitate object constancy).

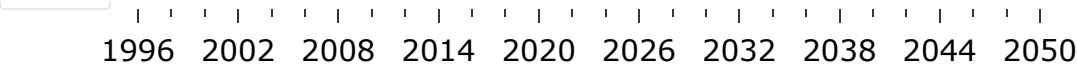
4.1 Building an animated bubble plot: `ggplotly()` method

In this section, you will learn how to create an animated bubble plot by using `ggplotly()` method.

```
gg <- ggplot(globalPop,
  aes(x = Old,
    y = Young,
    size = Population,
    colour = Country)) +
  geom_point(aes(size = Population,
    frame = Year),
    alpha = 0.7,
    show.legend = FALSE) +
  scale_colour_manual(values = country_colors) +
  scale_size(range = c(2, 12)) +
  labs(x = '% Aged',
    y = '% Young')

ggplotly(gg)
```





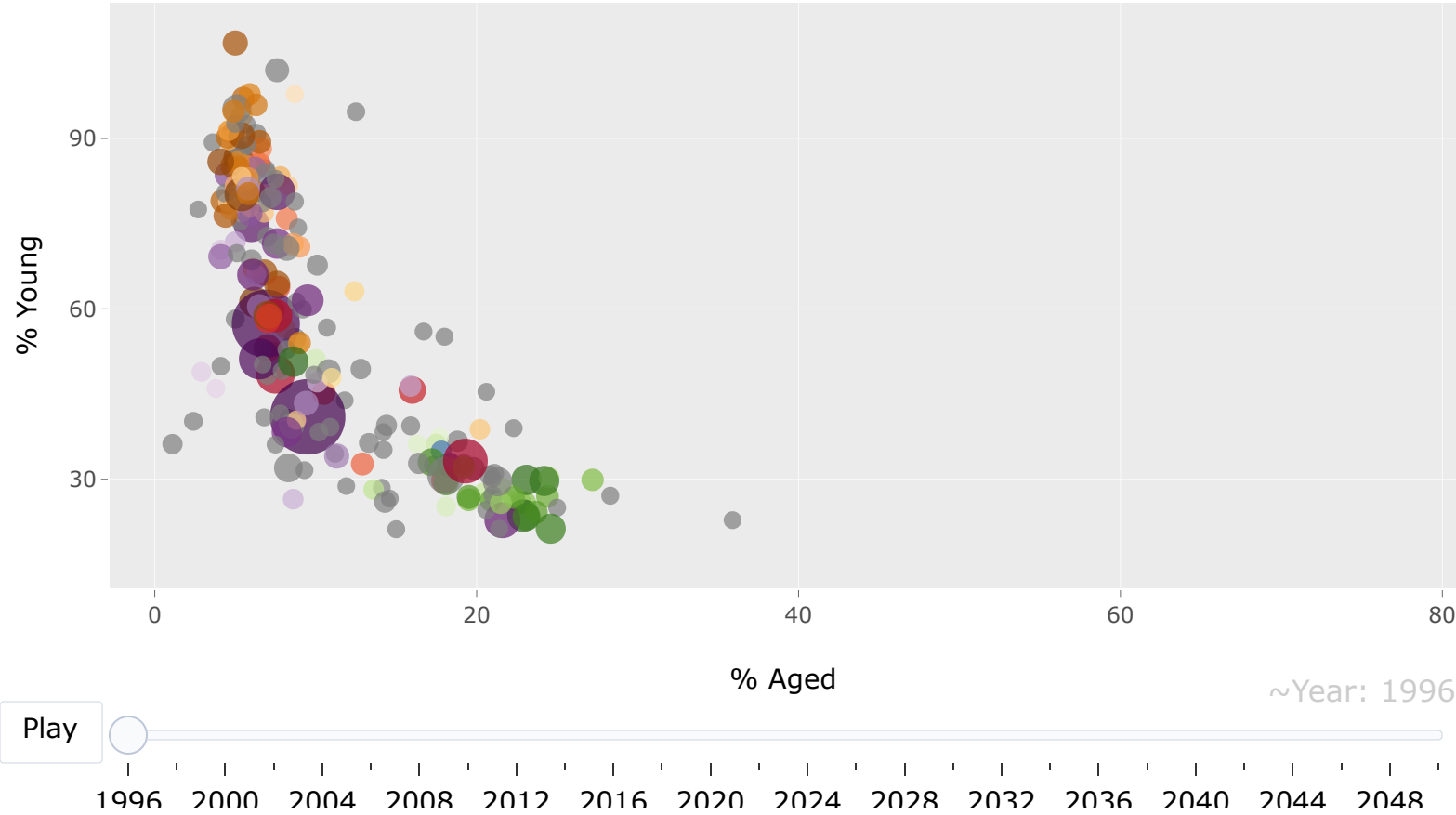
Things to learn from the code chunk above

- Appropriate ggplot2 functions are used to create a static bubble plot. The output is then saved as an R object called *gg*.
- `ggplotly()` is then used to convert the R graphic object into an animated svg object.

Notice that although `show.legend = FALSE` argument was used, the legend still appears on the plot. To overcome this problem, `theme(legend.position='none')` should be used as shown in the plot and code chunk below.

```
gg <- ggplot(globalPop,
  aes(x = Old,
      y = Young,
      size = Population,
      colour = Country)) +
  geom_point(aes(size = Population,
                 frame = Year),
             alpha = 0.7) +
  scale_colour_manual(values = country_colors) +
  scale_size(range = c(2, 12)) +
  labs(x = '% Aged',
       y = '% Young') +
  theme(legend.position='none')

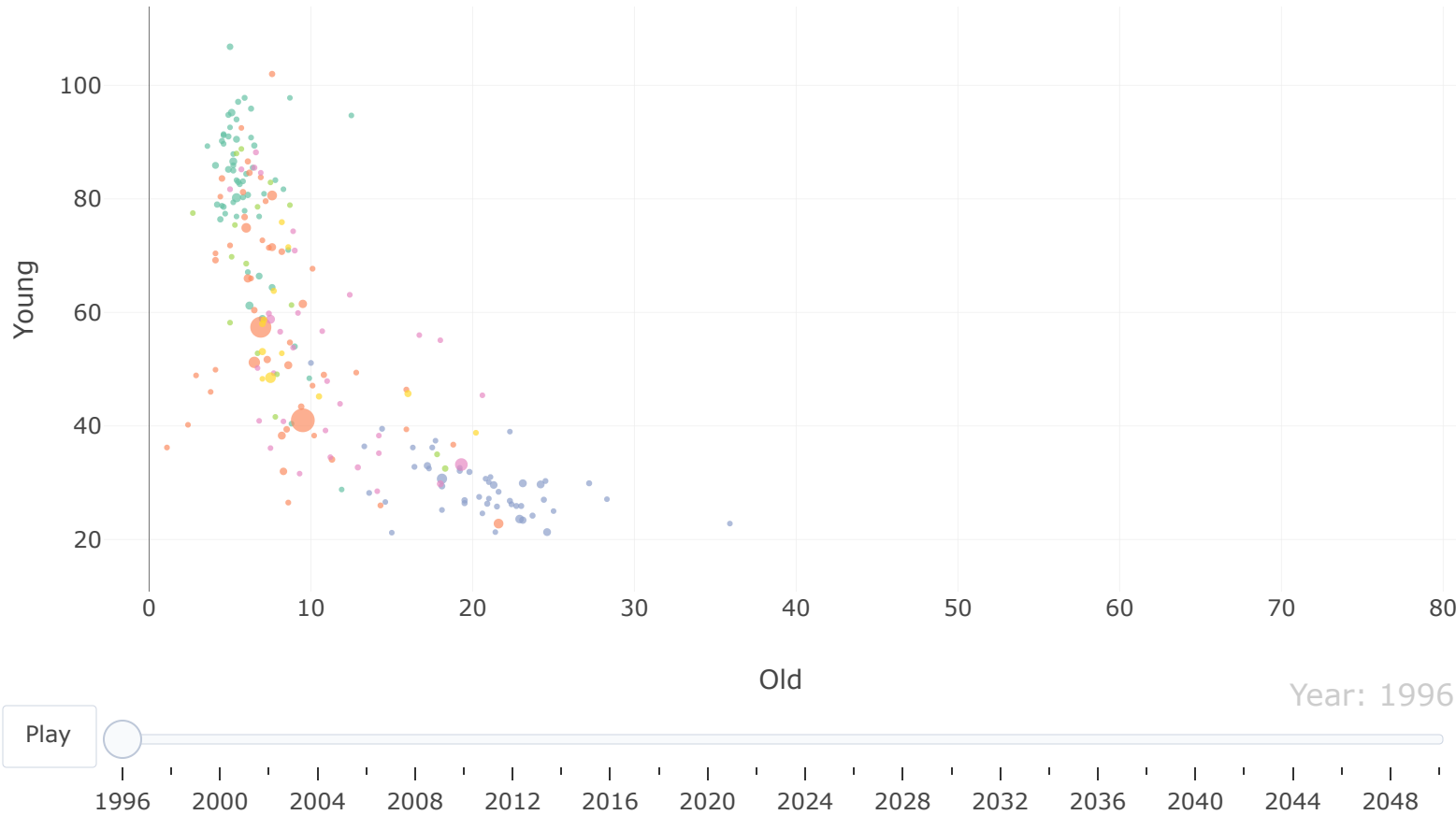
ggplotly(gg)
```



4.2 Building an animated bubble plot: `plot_ly()` method

In this sub-section, you will learn how to create an animated bubble plot by using `plot_ly()` method.

```
bp <- globalPop %>%
  plot_ly(x = ~Old,
          y = ~Young,
          size = ~Population,
          color = ~Continent,
          sizes = c(2, 100),
          frame = ~Year,
          text = ~Country,
          hoverinfo = "text",
          type = 'scatter',
          mode = 'markers'
        ) %>%
  layout(showlegend = FALSE)
bp
```



5 Reference

- Kam, T. S. (2024). *Programming Animated Statistical Graphics with R*. <https://r4va.netlify.app/chap04>
- [Getting Started](#)
- Visit this [link](#) for a very interesting implementation of gganimate by your senior.
- [Building an animation step-by-step with gganimate](#).
- [Creating a composite gif with multiple gganimate panels](#)